```
--------------------------------------------------------------------------------¶-- TLALOC¬(The Lonesome Ada Loving Ol'timer Compiler) Strict Ada 83 C
ompiler¶-- Copyright (C) 2024-2026 Vincent MORIN - Universit de Bretagne Occidentale¶--¶-- This program is free software: you can redistribute it and/
or modify¶-- it under the terms of the GNU General Public License as published by¶-- the Free Software Foundation, either¬version 3¬of the License, or
¶-- (at your option)¬any later¬version.¶--¶-- This program is distributed in the hope that it will be useful,¶-- but WITHOUT ANY WARRANTY; without eve
n the implied warranty of¶-- MERCHANTABILITY or FITNESS¬FOR A PARTICULAR PURPOSE. See¬the¶-- GNU General Public License¬for more details.¶--¶-- You sh
ould have received a¬copy of the GNU General Public License¶-- along with this program. If not, see¬<https://www.gnu.org/licenses/>.¶-----------------
---------------------------------------------------------------¶¶with DIRECT_IO;¶separate(¬IDL )¶¶----------------------------------------------------
--------------------------------------------------------------------¶--¬¬PAGE_MAN.ADB¬¬Vincent MORIN¶-------------------------------------------------
-----------------------------------------------------------------------¶--¬1¬2¬3¬4¬5¬6¬7¬8¬9¬A¬B¬C¶¶¬¬¬¬¬¬¬-------¶¬package body¬¬¬PAGE_MAN¶is¬¬¬¬¬----
----¶¶  package¬PAGE_IO¬¬is new DIRECT_IO( SECTOR );¶  use PAGE_IO;¶¶  subtype¬POSITIVE_COUNT¬is PAGE_IO.POSITIVE_COUNT;¶¶  WORK_FILE¬¬: PAGE_IO.FILE_
TYPE;¶  RPG_ALLOC¬¬: RPG_NUM¬¬¬:= RPG_NUM'LAST;¶¶  PAGE_MAN_DEBUG¬¬: BOOLEAN¬¬¬:= FALSE;¶¶--|---------------------------------------------------------
----------------------------------------¶¶¬¬¬¬¬¬INIT_PAGE_MAN¬--¶¶--|---------------------------------------------------------------------------------
----------------¶procedure¬INIT_PAGE_MAN is¶begin¶  ASSOC_PAGE := (others=> 0);¬¬¬¬¬¬¬¬--| POUR TOUTE PAGE¬VIRTUELLE, PAS DE PAGE REELLE¬ASSOCIEE¶  PA
G¬   := (others=> (¬VP¬¬=> 0,¬¬¬¬¬¬--| INITIALISER LA TABLE DES PAGES PHYSIQUES¶¬¬¬AREA¬¬=> 0,¶¬¬¬RECUPERABLE¬=> TRUE,¶¬¬¬CHANGED¬¬=> FALSE,¶¬¬¬DATA¬¬=
> new SECTOR )¶¬¬¬);¶  AREA¬   := (others=> (VP¬=> 0, FREE_LINE => LINE_NBR'LAST ) );¬¬¬¬¬--| INIT DES POINTS¬D INSERTION POUR FORCER UNE ALLOC LA PREM
IERE FOIS¶  RPG_ALLOC  := RPG_NUM'LAST;¶end;¶--|#################################################################################################¶¶--¬
¬##¬¬CREATE_PAGE_MANAGER¬¬¬##¶¶--|#################################################################################################¶procedure¬CREATE_P
AGE_MANAGER¬( PAGE_FILE_NAME :STRING ) is¶begin¶  INIT_PAGE_MAN;¬¬¬¬¬¬¬¬¬--| INITIALISER LES¬TABLES¶  PAGE_IO.CREATE ( WORK_FILE,¬INOUT_FILE, PAGE_FIL
E_NAME );¬¬¬¬¬¬¬--| CREATION DU FICHIER DE PAGES¶  HIGH_VPG := 0;¬¬¬¬¬¬¬¬¬¬¬--| PAS DE PAGE INITIALEMENT¶  CUR_VP¬ := MAX_VPG;¶¶if PAGE_MAN_DEBUG then PUT
_LINE( "page_man : create_page_manager" ); end if;¶¶exception¶  when others => PUT_LINE( "ERREUR CREATE_PAGE_MANAGER (PAGE_FILE_NAME = " & PAGE_FILE_N
AME & ")" );¶end;¶--|#################################################################################################¶¶--¬¬##¬¬OPEN_PAGE_MANAGER¬¬¬##
¶¶--|#################################################################################################¶procedure¬OPEN_PAGE_MANAGER (¬PAGE_FILE_NAME :S
TRING ) is¶begin¶  INIT_PAGE_MAN;¬¬¬¬¬¬¬¬¬¬¬--| INITIALISER LES¬TABLES¶  PAGE_IO.OPEN( WORK_FILE, INOUT_FILE, PAGE_FILE_NAME );¬¬¬¬¬¬--| OUVRIR LE FICHIE
R DE PAGES¶¶if PAGE_MAN_DEBUG then PUT_LINE( "page_man : open_page_manager" ); end if;¶¶  CUR_RP := READ_PAGE( 1 );¬¬¬¬¬¬¬¬¬¬--| LIRE LA RACINE (PAGE VI
RTUELLE 1) DANS LA PAGE REELLE DE No RENDU¶  CUR_VP := 1;¬¬¬¬¬¬¬¬¬¬¬--| VIRTUELLE COURANTE = 1¶  HIGH_VPG := VPG_IDX( PAG( CUR_RP ).DATA.all( 1 ).ABSS )
;¬¬¬¬¬¬--| NUMERO DE LA DERNIERE PAGE VIRTUELLE (AUSSI NUMERO DE BLOC FICHIER)¶exception¶  when PAGE_IO.NAME_ERROR =>¶    PUT_LINE( "OPEN_PAGE_MANAGER
: NAME_ERROR " &¬PAGE_FILE_NAME );¶  when others =>¶    PUT_LINE( "ERREUR OPEN_PAGE_MANAGER (PAGE_FILE_NAME = "¬& PAGE_FILE_NAME & ")" );¶end;¶--|----
---------------------------------------------------------------------------------------------¶¶¬¬¬--¬INDIQUE_RECUPERABLE¬¬--¶¶--|---------------------
----------------------------------------------------------------------------¶procedure¬INDIQUE_RECUPERABLE¬( RP :RPG_NUM ) is¬¬¬¬¬¬--| RENDRE LA PAGE 
REELLE FLOTTANTE (OU¬RECUPERABLE)¶  THE_PAGE¬: RPG_DATA¬renames PAG( RP );¶begin¶¶if PAGE_MAN_DEBUG then PUT_LINE( "page_man : indique_recuperable"¶¬&
 " rp=" & RPG_IDX'IMAGE( RP ) & "(area=" & area_idx'image(THE_PAGE.AREA)¶¬& " recup=" & BOOLEAN'IMAGE( THE_PAGE.RECUPERABLE¬) & ")"  );¶end if;¶¶  if
THE_PAGE.RECUPERABLE then return; end if;¬¬¬¬¬¬¬--| SI DEJA RECUPERABLE RIEN A FAIRE¶  if THE_PAGE.AREA > 0 then¬¬¬¬¬¬¬¬¬¬--| S IL Y A UN POINT D'INSERIO
N SUR LA¬PAGE REELLE¶    THE_PAGE.AREA¬     := 0;¬¬¬¬¬¬¬¬¬--| MAINTENANT PLUS¬DE ZONE INSERTION¶    THE_PAGE.CHANGED     := TRUE;¬¬¬¬¬¬¬¬¬--| INDIQUER CE
CHANGEMENT¶  end if;¶  THE_PAGE.RECUPERABLE := TRUE;¬¬¬¬¬¬¬¬¬--| INDIQUER QUE LA¬PAGE PHYSIQUE EST RECUPERABLE¶end INDIQUE_RECUPERABLE;¶--|-------------
------------------------------------------------------------------------------------¶¶¬¬¬¬¬¬WRITE_PAGE¬--¶¶--|----------------------------------------
---------------------------------------------------------¶procedure¬WRITE_PAGE ( RP :RPG_NUM ) is¶begin¶¶if PAGE_MAN_DEBUG then PUT_LINE( "page_man : 
write_page rp=" & RPG_IDX'IMAGE( RP )¶¬& "(vp=" & VPG_IDX'IMAGE( PAG( RP ).VP ) & ")"¶¬); end if;¶¶  PAGE_IO.WRITE( WORK_FILE, PAG( RP ).DATA.all, POS
ITIVE_COUNT( PAG( RP ).VP ) );¬¬--| ECRIRE LA PAGE¶  PAG( RP¬).CHANGED¬:= FALSE;¬¬¬¬¬¬¬¬¬¬--| ON VIENT DE SAUVER, PAS ENCORE DE MODIFICATION¶end;¶--|---
-----------------------------------------------------------------------------------------------¶¶¬¬¬¬¬FREE_PAGE¬¬--¶¶--|------------------------------
-------------------------------------------------------------------¶procedure¬FREE_PAGE¬( RP :RPG_NUM ) is¶  THE_PAGE¬: RPG_DATA¬renames PAG( RP );¶  
VP¬¬: VPG_IDX¬¬renames THE_PAGE.VP;¬¬¬¬¬--| INDICE DE LA PAGE VIRTUELLE ASSOCIEE A LA PAGE RP¶begin¶if PAGE_MAN_DEBUG then PUT_LINE( "page_man : free_p
age rp="¬& RPG_IDX'IMAGE( RP¬)¶¬& "(vp=" & VPG_IDX'IMAGE( VP )¶¬& " recuperable=" &¬BOOLEAN'IMAGE( THE_PAGE.RECUPERABLE ) &¬")" ); end if;¶¶  if VP =¬
0 then return; end if;¬¬¬¬¬¬¬¬¬--| RIEN A FAIRE SI¬LA PAGE PHYSIQUE EST NON LIEE¶  if THE_PAGE.CHANGED then¬¬¬¬¬¬¬¬¬¬--| SI LA¬PAGE PHYSIQUE A ETE¬MODIFIE
E¶    WRITE_PAGE( RP );¬¬¬¬¬¬¬¬¬¬--| LA SAUVEGARDER¶  end if;¶  ASSOC_PAGE( VP ) := 0;¬¬¬¬¬¬¬¬¬¬--| POUR LA VIRTUELLE VP : PLUS DE PAGE¬PHYSIQUE LIEE¶  VP
 := 0;¬¬¬¬¬¬¬¬¬¬¬--| POUR LA PHYSIQUE : PLUS DE PAGE VIRTUELLE LIEE¶end FREE_PAGE;¶--|------------------------------------------------------------------
-------------------------------¶¶¬¬¬¬¬ASSIGN_PAGE¬--¶¶--|---------------------------------------------------------------------------------------------
-----¶function ASSIGN_PAGE ( VP :VPG_NUM ) return RPG_NUM is¬¬¬¬¬¬--| ASSOCIER UNE PAGE VIRTUELLE A UNE REELLE¶begin¶¶if PAGE_MAN_DEBUG then PUT_LINE(
"page_man : start assign ----------"¶¬& " vp=" & VPG_IDX'IMAGE( VP ) ); end if;¶¶CHERCHE_ET_DEGAGE:¶  loop¶    if RPG_ALLOC < MAX_RPG then¬¬¬¬¬¬¬¬--| L
E POINTEUR D'ALLOCATION EST AU DESSOUS DE LA¬LIMITE¶      RPG_ALLOC := RPG_ALLOC + 1;¬¬¬¬¬¬¬¬--| LE MONTER¶    else¶      RPG_ALLOC := 1;¬¬¬¬¬¬¬¬¬¬--| SI
NON¬BOUCLER EN LE REMETTANT AU DEBUT¶    end if;¶¶if PAGE_MAN_DEBUG then PUT_LINE( "page_man :       assign  cherche_degage"¶¬& " rpg_alloc=" & RPG_ID

X'IMAGE( RPG_ALLOC )¶¬& " (vp="¬& VPG_IDX'IMAGE( PAG( RPG_ALLOC ).VP)¶¬& " recuperable=" &¬BOOLEAN'IMAGE( PAG(¬RPG_ALLOC¬).RECUPERABLE) & ")" );¶end i
f;¶¶    exit when PAG( RPG_ALLOC ).VP = 0 or else PAG( RPG_ALLOC ).RECUPERABLE;¬¬¬¬--| SORTIR : ON A UNE PAGE PHYSIQUE NON¬LIEE OU RECUPERABLE¶  end lo
op CHERCHE_ET_DEGAGE;¶¶  if PAG(¬RPG_ALLOC¬).RECUPERABLE then FREE_PAGE(¬RPG_ALLOC¬); end if;¬¬¬¬--| SI RECUPERABLE LIBERER¶  ASSOC_PAGE( VP )¬  := RPG
_ALLOC;¬¬¬¬¬¬¬¬¬--| INDIQUER LA PAGE PHYSIQUE¬RECUPEREE¬COMME ASSOCIEE A LA¬VIRTUELLE¶  PAG( RPG_ALLOC ).VP := VP;¬¬¬¬¬¬¬¬¬¬--| INDIQUER LA PAGE VIRTUELLE
 COMME ASSOCIEE A LA PHYSIQUE¶¶if PAGE_MAN_DEBUG then PUT_LINE( "page_man : ok assign -----"¶¬& " vp=" & VPG_IDX'IMAGE( VP )¶¬& " >> rp=" & RPG_IDX'IM
AGE( RPG_ALLOC ) ); end if;¶¶  return RPG_ALLOC;¶end ASSIGN_PAGE;¶--|###################################################################################
################¶¶--¬¬##¬¬READ_PAGE¬¬¬##¶¶--|#########################################################################################################
##############################################################################################################################################¶  funct
ion READ_PAGE ( VP :VPG_NUM ) return RPG_NUM is¶    RP¬¬: RPG_IDX¬¬renames ASSOC_PAGE(¬VP );¬¬¬¬¬¬--| INDICE DE PAGE PHYSIQUE LIEE A LA VIRTUELLE¶  begi
n¶¶if PAGE_MAN_DEBUG then PUT_LINE( "page_man : start read_page ----------"¶¬& " vp=" & VPG_IDX'IMAGE( VP )¶¬& " rp=" & RPG_IDX'IMAGE( RP ) ); end if;
¶¶    if RP¬= 0 then¬¬¬¬¬¬¬¬¬¬¬--| SI PAS DE PHYSIQUE ASSOCIEE¶      RP := ASSIGN_PAGE( VP );¬¬¬¬¬¬¬¬¬--| ASSOCIER UNE NOUVELLE PAGE¶      PAGE_IO.READ(¬W
ORK_FILE, PAG(RP).DATA.all, POSITIVE_COUNT( VP )¬);¬¬¬¬--| LIRE LE BLOC DE¬LA PAGE PHYSIQUE ASSOCIEE A LA VIRTUELLE¶    end if;¶¶if PAGE_MAN_DEBUG then
 PUT_LINE( "page_man : ok read_page ----------"¶¬& " vp=" & VPG_IDX'IMAGE( VP )¶¬& " rp=" & RPG_IDX'IMAGE( RP ) ); end if;¶¶    return RP;¶  end READ_
PAGE;¶--|##################################################################################################¶¶--¬¬##¬¬NEW_BLOCK¬¬¬##¶¶--|##############
##################################################################################¶  procedure NEW_BLOCK is¶  begin¶    AREA(¬1 ).FREE_LINE := LINE_NB
R'LAST;¬¬¬¬¬¬¬¬--| FORCERA UNE ALLOCATION DE¬BLOC PAR MANQUE DE LIGNE¶  end NEW_BLOCK;¶--|############################################################
######################################¶¶--¬¬##¬¬ALLOC_PAGE¬¬##¶¶--|###################################################################################
###########¶  procedure ALLOC_PAGE ( AR :AREA_IDX; REQUESTED_SIZE :LINE_NBR ) is¶    NB_FREE_LINES¬: LINE_NBR¬:= LINE_NBR( LINE_IDX'LAST ) - AREA( AR¬
).FREE_LINE + 1;¬--| NOMBRE DE LIGNES LIBRES AU POINT D INSERTION¶  begin¶    if NB_FREE_LINES >= REQUESTED_SIZE then¬¬¬¬¬¬¬¬--| IL Y A ASSEZ DE¬PLACE P
OUR CE LIEU D'INSERTION¶¶      declare¶        VP¬: VPG_IDX¬¬renames AREA( AR ).VP;¬¬¬¬¬--| LE NUM DE PAGE VIRTUELLE DU POINT D¬INSERTION¶        RP¬:
RPG_IDX¬¬renames ASSOC_PAGE(¬VP );¶        begin¶          if RP = 0 then¬¬¬¬¬¬¬¬¬¬--| LA PAGE REELLE A ETE LIBEREE¶¬RP := READ_PAGE( VP¬);¶¶if PAGE_MAN_DEB
UG then PUT_LINE( "page_man : alloc_page read_page"¶¬& " vp=" & VPG_IDX'IMAGE( VP )¶¬& " rp=" & RPG_IDX'IMAGE( RP ) ); end if;¶¶          end if;¶¶
   declare¶¬THE_PAGE¬: RPG_DATA¬renames PAG( RP );¶          begin¶¬if THE_PAGE.RECUPERABLE then¬¬¬¬¬¬¬¬¬--| SI PAGE INDIQUEE RECUPERABLE¶¶if PAGE_MAN_DEB
UG then PUT_LINE( "page_man : alloc recuperation"¶¬& " vp=" & VPG_IDX'IMAGE( VP )¶¬& " rp=" & RPG_IDX'IMAGE( RP ) ); end if;¶¶¬  THE_PAGE.RECUPERABLE
:= FALSE;¬¬¬¬¬¬¬¬--| REFIXER¶¬  THE_PAGE.AREA¬   := AR;¬¬¬¬¬¬¬¬¬--| REMETTRE LE POINT D INSERTION¶¬end if;¶¬return;¬¬¬¬¬¬¬¬¬¬¬--| RIEN A FAIRE DE¬PLUS (ON N
E PREND PAS L ESPACE ICI)¶          end;¶       end;¶¶    else¬¬¬¬¬¬¬¬¬¬¬--| PLACE¬INSUFFISANTE ALLOUER¶¶LACHE_PAGE_REELLE_TROP_PLEINE:¶     declare¶
VP¬¬: VPG_IDX¬¬:= AREA( AR ).VP;¬¬¬¬¬¬--| LE NUM DE PAGE VIRTUELLE DU POINT D¬INSERTION¶    begin¶      if VP /= 0 and then ASSOC_PAGE( VP ) /= 0 then¬
¬¬¬¬¬--| PAGE ASSOCIEE ET EN MEMOIRE POUR LE¬POINT D INSERTION¶          INDIQUE_RECUPERABLE( ASSOC_PAGE( VP ) );¬¬¬¬¬¬¬¬--| PAGE DEVENANT RECUPERABLE¶
  end¬if;¶    end LACHE_PAGE_REELLE_TROP_PLEINE;¶¶    begin¶      HIGH_VPG := HIGH_VPG + 1;¬¬¬¬¬¬¬¬¬--| UNE PAGE VIRTUELLE DE PLUS¶    exception¶      w
hen CONSTRAINT_ERROR =>¬¬¬¬¬¬¬¬¬--| SI ON¬DEPASSE¶        TEXT_IO.PUT_LINE ( "EXCES DE PAGES VIRTUELLES ! PLUS DE " & VPG_NUM'IMAGE( HIGH_VPG ) );¬--| M
ESSAGE¶        CLOSE_PAGE_MANAGER;¬¬¬¬¬¬¬¬¬¬--| FERMER LE FICHIER¶    end;¶¶ASSIGNE_PAGE_REELLE:¶    declare¶      RP¬¬: RPG_NUM¬¬:= ASSIGN_PAGE( HIGH_V
PG );¬¬¬¬¬--| ASSOCIER LA NOUVELLE PAGE¬VIRTUELLE¶    begin¶      PAG( RP ).DATA.all := (others=> TREE_VIRGIN);¬¬¬¬¬¬--| INITIALISER LE BLOC ALLOUE¶
  PAG( RP ).AREA     := AR;¬¬¬¬¬¬¬¬¬--| No DE¬LIEU D'INSERTION MENTIONNE¶      PAG( RP ).CHANGED  := TRUE;¶    end ASSIGNE_PAGE_REELLE;¶¶    AREA(¬AR )
:= (¬VP¬=> HIGH_VPG,¬¬¬¬¬¬¬--| INDICE DE PAGE VIRTUELLE¶¬¬FREE_LINE¬=> LINE_NBR'FIRST¬¬¬¬¬¬¬--| LIGNE¬UTILISABLE¶¬¬);¶  end if;¶end ALLOC_PAGE;¶--|#####
############################################################################################¶¶--¬¬##¬¬CLOSE_PAGE_MANAGER¬¬¬##¶¶--|####################
#############################################################################¶procedure¬CLOSE_PAGE_MANAGER is¶begin¶¶  declare¶    RP¬¬: RPG_NUM¬¬:= R
EAD_PAGE( 1 );¬¬¬¬¬¬--| AMENER LA PAGE VIRTUELLE 1¶  begin¶    if PAG( RP ).DATA.all(1).ABSS /= POSITIVE_SHORT( HIGH_VPG ) then¬¬¬¬¬¬--| SI LE¬NOMBRE DE¬
PAGES N'A¬PAS LA BONNE VALEUR¶      PAG( RP ).DATA.all(1).ABSS := POSITIVE_SHORT( HIGH_VPG );¬¬¬¬¬¬--| CHANGER LA VALEUR¶      PAG( RP ).CHANGED := TRUE
;¬¬¬¬¬¬¬¬¬--| MENTIONNER LE CHANGEMENT¶    end if;¶  end;¶¶--    PUT_LINE( "TREE_ROOT=" );    IDL.PRINT_NOD.PRINT_NODE( TREE_ROOT );¶--    PUT_LINE( "US
ER_ROOT=" );    IDL.PRINT_NOD.PRINT_NODE( D( XD_USER_ROOT, TREE_ROOT )¬);¶¶  for I in RPG_NUM loop FREE_PAGE( I );¬end loop;¬¬¬¬¬¬¬--| LIBERER LES PAGE
S (INCLUT¬UNE ECRITURE SI NECESSAIRE)¶¶if PAGE_MAN_DEBUG then PUT_LINE( "page_man : close_page_manager" ); end if;¶¶  PAGE_IO.CLOSE( WORK_FILE );¬¬¬¬¬
¬¬¬--| FERMER LE FICHIER DE PAGES¶  end¬CLOSE_PAGE_MANAGER;¶¬------------------¶¶¬¬¬-================--¶  procedure¬¬DELETE_PAGE_MANAGER¶  is¶  begin¶
    PAGE_IO.DELETE(¬WORK_FILE¬);¬¬¬¬¬¬¬¬¬--| DETRUIRE LE FICHIER DE PAGES (EN FIN DE WRITELIB)¶¶  end¬DELETE_PAGE_MANAGER;¶¬----------------¶¶¶¬------
---¶end¬PAGE_MAN;¶¬---------¶¶--¬1¬2¬3¬4¬5¬6¬7¬8¬9¬A¬B¬C¶---------------------------------------------------------------------------------------------
--------------------------¶
```